TravelPlanner 알고리즘 응용 설명

원래 알고리즘을 여행 일정 문제에 어떻게 맞추었는가

작성일: 2026-10-09 · 현재 로컬 구현 분석 기준

이 문서는 파일별 기능 목록 대신, 알고리즘의 원래 목적과 여행 앱에서 필요한 가공을 설명합니다. 각 절은 원래 원리, 그대로 적용할 때의 문제, 앱에 맞춘 변경, 동작 예시, 현재 사용 여부의 순서로 구성했습니다. 예시에 등장하는 거리와 금액은 원리를 보여주기 위한 가정이며 실제 장소의 운임이나 거리 자료가 아닙니다.

현재 일정 생성은 도시 방문 순서를 먼저 정하고, 각 날짜 안에서는 시간 조건을 만족하는 장소를 순차적으로 선택하는 구조입니다. 도시 순서에는 부분집합 동적 계획법을 사용하고, 하루 일정에는 제약 기반 탐욕 선택을 사용합니다. 2-opt 구현도 남아 있지만 현재 생성 경로에서 호출되지 않습니다. 따라서 현재 동작을 설명할 때에는 2-opt를 적용했다고 표현하지 않습니다.

1. 2-opt: 경로 교환으로 이동 거리를 줄이는 방법

원래 알고리즘의 역할

2-opt는 이미 만들어진 방문 순서에서 두 연결 구간을 골라 연결 방식을 바꾸는 국소 탐색 알고리즘입니다. 예를 들어 A → B → C → D에서 A-B와 C-D 연결 대신 A-C와 B-D를 연결하면, 가운데 방문 순서는 반대로 바뀌어 A → C → B → D가 됩니다. 바꾼 경로가 더 짧으면 채택하고 같은 검사를 반복합니다. 교차하거나 크게 돌아가는 경로를 줄이는 데 도움이 되지만 모든 가능한 순서를 비교하지 않으므로 전역 최단 경로를 보장하지는 않습니다.

```
교환 전 비교 거리 = 거리(A,B) + 거리(C,D)
교환 후 비교 거리 = 거리(A,C) + 거리(B,D)
교환 후가 더 짧으면 가운데 방문 순서를 뒤집는다.
```

여행 앱에 그대로 적용할 때의 문제

여행에서는 거리가 줄어도 식당 영업시간을 놓치거나, 예약한 고정 일정을 지나치거나, 숙소 도착 시각을 넘길 수 있습니다. 또한 입국 공항과 출국 공항의 역할이 다르므로 모든 지점을 자유롭게 뒤집어서는 안 됩니다. 거리 개선과 일정 실행 가능성은 별개의 조건입니다.

우리 코드에 남아 있는 가공 방식

기존 구현은 먼저 장소를 날짜별로 분할하고, 가까운 장소를 순서대로 연결하는 최근접 이웃 방식으로 초기 경로를 만듭니다. 이후 첫 지점과 마지막 지점이 바뀌지 않도록 내부 구간만 2-opt 교환 대상으로 삼습니다. 두 연결 구간의 직선 거리 합이 0.001km보다 많이 줄어들 때만 교환 후보로 인정합니다.

교환 후보가 생기면 임시 경로의 누적 직선 거리를 다시 계산해 15km를 넘는 후보를 거부합니다. 코드에서 이 검사에 시간 창 위반이라는 이름을 사용하지만, 실제로 검사하는 값은 영업시간이나 예약 시각이 아니라 거리입니다. 공항 항목은 이 재배열 뒤에 첫날 시작 또는 마지막 날 끝에 배치하는 방식입니다.

가공 결과를 설명하는 예시

기존 두 연결 구간의 합이 8km이고 교환 후 합이 3km라면 5km를 줄이는 후보입니다. 하지만 임시 경로 전체가 16km가 되면 현재 남아 있는 15km 제한에 걸려 교환을 채택하지 않습니다. 전체가 12km라면 이 거리 검사는 통과합니다. 이 검사를 통과했다는 사실만으로 점심 예약이나 숙소 마감을 지킨다고 설명할 수는 없습니다.

실제 사용 여부와 설명 가능한 범위

현재 생성 요청은 이 기존 2-opt 함수가 아니라 뒤에서 설명하는 하루 일정 시뮬레이션을 사용합니다. 따라서 정확한 설명은 "기존에는 거리 개선을 위한 2-opt에 내부 구간 교환과 총거리 제한을 붙인 구현이 있었으며, 현재 생성에서는 시간 제약을 직접 검사하는 후보 선택 방식을 사용한다"입니다. 2-opt에 영업시간과 식사 제약을 붙여 현재 사용 중이라고 쓰면 실제 구현과 다릅니다.

2. Held-Karp 계열: 도시 방문 순서를 공항 조건에 맞추기

원래 알고리즘의 역할

외판원 문제는 여러 도시를 방문하는 순서 가운데 이동 비용이 작은 순서를 찾는 문제입니다. Held-Karp 계열의 부분 집합 동적 계획법은 "어떤 도시를 이미 방문했는가"와 "현재 마지막 도시가 어디인가"를 하나의 상태로 저장합니다. 같은 상태에 도달하는 여러 순서 중 비용이 가장 작은 것을 남겨 중복 계산을 줄입니다. 순환형 외판원 문제에서는 마지막에 처음 도시로 돌아가는 비용까지 계산합니다. 원형 참고 문헌: Held와 Karp의 동적 계획법 논문 (https://epubs.siam.org/doi/10.1137/0110015).

여행 앱에 그대로 적용할 때의 문제

우리 사용자는 입국 공항과 출국 공항을 따로 선택합니다. 출발한 도시로 반드시 돌아오는 순환 경로를 만들면 불필요한 왕복이 생길 수 있습니다. 또한 신치토세로 들어왔다고 해서 사용자가 선택하지 않은 삿포로를 자동으로 첫 방문 도시로 넣는 것은 사용자의 선택과 다릅니다.

우리 앱에 맞춘 가공

우선 사용자가 선택한 도시만 방문 대상 집합으로 만듭니다. 도시의 대표 위치는 DB 장소 좌표에서 위도와 경도의 중앙값을 각각 구해 만듭니다. 입국 공항에서 가장 가까운 선택 도시를 첫 도시로 고정하고, 나머지 도시의 순서를 부분 집합 동적 계획법으로 비교합니다. 마지막 도시를 결정할 때에는 그 도시에서 출국 공항까지의 거리도 더합니다. 처음 도시로 되돌아오는 연결은 추가하지 않습니다.

```
상태 = 방문한 도시 집합 + 마지막 도시
다음 상태 비용 = 현재 상태 비용 + 마지막 도시→다음 도시 거리
최종 비교 비용 = 모든 도시 방문 비용 + 마지막 도시→출국 공항 거리
```

첫 도시는 공항과의 근접성으로 미리 고정하므로, 이 알고리즘이 비교하는 범위는 "고정된 첫 도시에서 시작하는 순서"입니다. 입국 공항에서 첫 도시까지의 거리는 이 범위의 모든 후보에 동일하므로 상태 비용에서 생략할 수 있습니다. 모든 첫 도시 후보까지 자유롭게 비교하는 문제와는 다릅니다.

가공 결과를 설명하는 예시

입국 공항과 가까운 선택 도시가 A이고 다른 선택 도시가 B, C라면 A → B → C와 A → C → B를 비교합니다. 앞의 도시 이동만 짧은 순서가 아니라 마지막 도시에서 출국 공항까지 합친 거리가 더 짧은 순서를 선택합니다. 선택하지 않은 도시 X는 공항 옆에 있어도 방문 대상에 추가하지 않습니다.

담당 기능과 한계

이 방법은 도시 단위의 큰 방문 순서를 담당합니다. 순서를 정한 뒤 같은 도시의 날짜가 연속되도록 여행 일수에 배정합니다. 현재 일차 배정은 순서에 따른 대략적인 균등 배정이며, 관광지 수나 실제 운행 시간에 비례해 체류일을 최적화하는 방식은 아닙니다. 거리도 Haversine 직선 거리이므로 실제 철도망 최단 경로와 다를 수 있습니다. 도시 수는 계산량을 제한하기 위해 최대 15개로 둡니다.

3. 탐욕 선택: 가장 가까운 장소 대신 지금 방문 가능한 장소 선택

원래 알고리즘의 역할

최근접 이웃 방식은 현재 위치에서 가장 가까운 미방문 지점을 다음 방문지로 선택하고 이 과정을 반복합니다. 탐욕 알고리즘은 이처럼 매 단계에서 정한 평가 기준에 가장 유리한 후보 하나를 고릅니다. 전체 경우를 모두 탐색하는 것보다 계산을 줄일 수 있지만, 지금 유리한 선택이 하루 전체에서도 가장 좋은 선택이라는 보장은 없습니다.

여행 앱에 그대로 적용할 때의 문제

가장 가까운 관광지가 아직 문을 열지 않았을 수 있고, 가까운 식당을 계속 고르면 식사 간격이 너무 짧아집니다. 마지막 관광지까지 갈 수 있더라도 숙소로 돌아갈 시간은 부족할 수 있습니다. 고정 일정에 늦거나 긴 관광 때문에 저녁 식사를 못 넣는 상황도 생깁니다.

우리 앱에 맞춘 가공

후보를 고르기 전에 현재 시각에서 이동해 도착할 시간, 영업 시작까지 기다리는 시간, 체류시간과 여유시간을 계산합니다. 장소의 해당 날짜 영업시간 안에 방문을 끝낼 수 있어야 하며, 방문을 마친 뒤 숙소 또는 출국 공항까지 이동해 하루 마감을 지킬 수 있어야 합니다. 고정 일정이 있으면 그 장소까지 이동할 시간도 포함해 충돌을 검사합니다. 충돌하면 고정 일정 종료 후를 기준으로 방문 가능 시각을 다시 계산합니다.

식사는 기본 2회이며 일반 여행은 시작 시각 사이를 최소 300분으로 유지합니다. 맛집 테마에서 5끼를 선택하면 간격을 150분으로 줄입니다. 간식은 식사와 별도로 횟수와 간격을 관리합니다. 첫날 입국 수속 120분, 마지막 날 출국 준비 180분을 먼저 활동 범위에서 반영합니다. 이동시간의 20% 여유도 일정 배치에서 사용합니다.

조건을 통과한 후보 가운데 방문 시작까지의 시간, 이동시간, 우회 페널티, 날씨 점수로 계산한 비용이 작은 후보를 고릅니다. 식사 시작이 현재 시각으로부터 60분 안에 잡히는 경우에는 식사 후보를 우선합니다. 이어서 관광을 선택하면 예정 식사에 도착하지 못하는지도 확인하고, 그런 경우 짧은 관광으로 바꾸거나 식사를 먼저 배치합니다.

가공 결과를 설명하는 예시

현재 10:00이고, 가까운 관광지 A는 12:00에 열며 조금 먼 관광지 B는 이미 열었다고 가정합니다. 단순 최근접 이웃 방식은 A를 고릅니다. 우리 방식은 A의 대기시간을 비용에 반영하고 B의 방문·복귀 가능성을 함께 비교합니다. 식사 예정 시각이 11:00인데 B의 체류시간이 2시간이라면 식사를 놓치는 조건까지 확인해 B를 바로 넣지 않을 수 있습니다.

담당 기능과 한계

이 방식은 하루에 어느 장소를 몇 시에 방문할지 결정합니다. 짧은 거리만 만족하는 경로를 만드는 대신, 시간 제약을 만족하는 일정 후보를 구성합니다. 식사 횟수는 목표지만 모든 시간 제약과 항상 함께 만족할 수 있는 보장은 없습니다. 선택한 활동 시간이 너무 짧다면 영업시간·식사 간격·공항 마감을 지키기 위해 방문 수가 줄어들 수 있습니다.

4. 삼각 부등식 페널티: 되돌아가는 후보를 선택 단계에서 억제

원래 수학적 성질과 목적

삼각 부등식은 A에서 C로 직접 가는 거리보다 A → B → C로 거쳐 가는 거리가 짧을 수 없다는 성질입니다. 이 차이를 구하면 B를 거치면서 얼마나 우회했는지 나타낼 수 있습니다. 이는 2-opt처럼 이미 만든 순서를 뒤집는 알고리즘이 아니라 후보를 평가할 때 쓰는 비용 항입니다.

우리 앱에 맞춘 가공

직전 두 방문 장소와 다음 후보를 사용합니다. A는 두 번째로 최근에 방문한 장소, B는 현재 위치인 직전 장소, C는 다음 후보입니다. 거리(A,B) + 거리(B,C) - 거리(A,C)를 계산하고, 음수가 되지 않게 한 뒤 3을 곱해 후보 비용에 더합니다. 이때의 C는 숙소나 복귀 지점이 아니라 다음 후보 장소입니다. 숙소 복귀 여부는 별도의 시간 제약 검사입니다.

```
우회 페널티 = max(0, 거리(A,B)+거리(B,C)-거리(A,C)) × 3
선택 비용 = 방문 시작 시각-현재 시각 + 이동시간×2
          + 우회 페널티 - 날씨 적합도
```

가공 결과를 설명하는 예시

A에서 B까지 북쪽으로 5km 이동한 상태라고 가정합니다. C가 B에서 다시 남쪽으로 4km 떨어져 있고 A에서 C까지 1km라면 우회량은 5+4-1=8km이며 비용에 24를 더합니다. 다른 후보 D가 같은 진행 방향으로 2km 더 떨어져 있고 A에서 D까지 7km라면 우회량은 5+2-7=0km입니다. 나머지 조건이 같다면 D가 유리합니다.

담당 기능과 한계

일단 방문 순서를 만들고 수정하는 것이 아니라, 다음 후보를 선택할 때 큰 역방향 이동을 덜 선택하도록 합니다. 점수의 3과 이동시간의 2는 앱에서 정한 경험적 계수이며 학습으로 얻은 값이 아닙니다. 첫 두 위치가 없거나 좌표가 없으면 이 페널티를 계산하지 않습니다. 먼 미래의 전체 경로까지 보는 방식도 아니므로 모든 지그재그 이동을 없앤다고 보장할 수는 없습니다.

5. 가중 점수: 사용자 취향을 후보 선별 기준으로 바꾸기

원래 방식의 역할

가중 점수 방식은 여러 평가 항목에 점수를 부여하고 합산해 후보를 비교합니다. 거리 하나만으로 고르는 것과 달리 선호도, 활동 적합성 같은 여러 기준을 함께 사용할 수 있습니다. 어떤 항목은 점수보다 강한 조건으로 처리해 아예 후보에서 제외할 수도 있습니다.

여행 앱에 그대로 적용할 때의 문제

많은 관광지를 확보해도 사용자가 원하는 맛집이나 쇼핑 일정이 부족할 수 있습니다. 반대로 선호 테마 점수를 크게 주는 것만으로는 사용자가 제외한 활동을 확실히 막지 못합니다. 가족 여행과 혼자 여행에 같은 우선순위를 적용하는 것도 적합하지 않을 수 있습니다.

우리 앱에 맞춘 가공

기본 50점에 선호 테마가 일치하면 테마별로 60점을 더합니다. 제외 테마는 점수를 낮추는 대신 후보에서 제거합니다. 가족 여행에서는 관광지에 40점을 더하고 활동형 장소에 50점을 감점합니다. 도시 중심에서 10km를 넘으면 초과 1km당 5점을 감점합니다. 점수가 양수인 후보를 남기고 점수 순으로 정렬합니다.

가공 결과를 설명하는 예시

사용자가 쇼핑을 선호하면 다른 조건이 같은 쇼핑 장소는 기본 50점에 60점을 더해 110점이 됩니다. 제외 테마에 해당하면 좋은 위치나 높은 기본 점수가 있어도 후보에서 빠집니다. 중심에서 15km 떨어져 있으면 거리 항목에서 25점을 감점합니다. 이 예시는 한 항목의 작동을 보여주며 실제 점수는 적용되는 항목의 합입니다.

담당 기능과 한계

후보 풀을 사용자의 여행 성격에 맞추는 역할입니다. 이 점수는 앞 절의 하루 방문 비용식에 그대로 더해지는 값이 아닙니다. 선별과 정렬에 영향을 주고, 실제 방문 여부와 시각은 시간 제약을 적용한 탐욕 선택이 결정합니다. 따라서 테마 점수가 가장 높다고 항상 첫 방문지가 되는 것은 아닙니다.

6. 기간 선택 동적 계획법: 전국 패스 한 장 대신 사용 기간 조합 비교

원래 알고리즘의 역할

가중 구간 선택 문제는 시작·종료가 있는 선택지와 각 선택지의 이득이 주어졌을 때, 겹치지 않는 선택지들의 이득 합이 큰 조합을 고르는 문제입니다. 동적 계획법은 어떤 구간을 선택하는 경우와 선택하지 않는 경우를 비교하고, 이전 기간에서 구한 최선의 결과를 재사용합니다.

여행 앱에 그대로 적용할 때의 문제

패스 이름이나 가격만 비교하면 그 패스가 사용자의 실제 노선에 적용되는지 알 수 없습니다. 7일 패스를 하루씩 끊은 상품처럼 계산하거나 여러 패스가 같은 열차 요금을 반복 할인하면 잘못된 절약액이 나옵니다. 전국 패스가 비싸도 지역 패스의 조합이 더 저렴할 수 있습니다.

우리 앱에 맞춘 가공

먼저 실제 일정의 이동 요금을 모으고, 각 패스가 적용되는 운영사·노선·도시·제외 열차를 검사합니다. 적용 구간에 해당하는 일반 요금 합에서 패스 한 장 가격을 빼 절약액을 만듭니다. 연속권은 시작일과 연속 유효기간을 가진 후보로 만듭니다. 일자별로 이전까지의 최선 조합과 그날 끝나는 패스를 추가한 조합을 비교합니다. 절약하지 못하는 패스는 자동 추천 조합에 넣지 않습니다.

```
패스 후보의 이득 = 적용 구간 일반 요금 - 패스 한 장 가격
오늘까지의 최선 = 이전 날 최선과
                  패스 시작 전 최선 + 해당 패스 이득 중 큰 값
```

가공 결과를 설명하는 예시

1~5일에 적용되는 지역 패스 A의 절약액이 3,000엔이고 8~10일의 지역 패스 B가 2,000엔을 절약한다고 가정합니다. 해당 기간을 넓게 덮는 다른 패스의 절약액이 4,000엔이라면 A와 B를 함께 쓰는 조합의 5,000엔이 유리합니다. 반면 A와 B가 같은 날짜 구간을 차지하면 이 기간 선택 단계에서는 함께 채택하지 않습니다.

담당 기능과 한계

언제 어느 기간 패스를 쓰는지 결정하는 역할입니다. 7일권은 7일 연속 유효기간과 한 장 가격을 유지합니다. 여행이 3일이라도 7일권 가격을 3일치로 잘라 계산하지 않습니다. 현재 철도 기간 조합을 먼저 고른 뒤 다른 패스를 처리하는 구조이므로, 모든 종류의 패스를 동시에 비교하는 전역 최저가 보장은 없습니다. 또한 서로 다른 운영사 패스라도 이 단계에서 기간이 겹치면 함께 선택할 수 없어 최적 조합을 놓칠 수 있습니다.

7. 집합 탐색과 메모이제이션: 하루 지역 패스의 중복 할인 방지

원래 알고리즘의 역할

집합 조합 탐색은 각각의 선택지가 포함하는 항목 집합을 보고, 서로 충돌하지 않는 선택지 조합을 찾는 방식입니다. 메모이제이션은 같은 상태의 결과를 저장해 다시 계산하지 않는 기법입니다. 여기서는 이미 할인한 이동 구간과 현재 검사 중인 후보 위치가 상태가 됩니다.

여행 앱에 맞춘 가공

일정의 철도·버스 이동마다 번호를 매깁니다. 각 지역 패스가 실제로 할인할 수 있는 이동 번호를 비트 집합으로 기록합니다. 패스를 추가할 때 이미 선택한 패스와 같은 이동 번호가 겹치는지 검사합니다. 겹치면 중복 할인을 막기 위해 함께 선택하지 않고, 겹치지 않으면 절약액 합을 비교합니다. 서로 관련 없는 날짜·노선 집합은 나누어 계산하고, 같은 적용 구간을 갖는 후보는 더 저렴한 것을 남깁니다.

도쿄의 패스라고 도쿄 내 모든 철도를 할인하지 않습니다. 운영사와 노선 조건을 확인한 이동만 적용 집합에 넣습니다. 확인된 사용 시각이 있는 시간제 패스는 첫 사용 시각에서 유효시간을 계산합니다. 당일권은 같은 날짜 이동을 기준으로 검사합니다. 정보가 부족한 구간은 적용 대상으로 추정하지 않습니다.

가공 결과를 설명하는 예시

하루 지하철 구간의 일반 요금이 1,200엔이고 패스 A가 그 구간을 800엔에 덮는다면 400엔을 절약합니다. 같은 구간을 덮는 패스 B가 900엔이면 A와 B를 둘 다 사서 중복 할인하지 않고 A를 남깁니다. 별도의 버스 구간 일반 요금이 700엔이고 버스 패스 C가 500엔이며 A와 적용 구간이 겹치지 않는다면 A+C로 합계 600엔을 절약하는 조합을 검토할 수 있습니다.

일별 비용 표시로 연결한 가공

선택한 패스의 적용 집합을 일차별로 다시 나누어 일반 요금과 실제 차감할 요금을 구합니다. 패스 구매 가격은 시작일에 한 번만 더하고, 나머지 날에는 그날의 적용 구간 차감만 반영합니다.

```
일별 패스 적용 금액 = 그날 일반 요금
                - 그날 패스로 덮는 요금
                + 그날 새로 구매하는 패스 가격
```

예를 들어 이틀간 사용 가능한 패스 한 장이 1,500엔이고 적용 구간 요금이 첫날 1,000엔, 둘째 날 1,200엔이라면 첫날은 구매 금액 1,500엔, 둘째 날은 적용 구간에 추가 결제가 없어 0엔으로 표시합니다. 합계는 1,500엔이며, 매일 패스 가격을 더한 3,000엔이 아닙니다. 적용되지 않는 이동이 있으면 해당 날짜에 일반 요금으로 남깁니다. 이는 사용일별 비용 평균이 아니라 구매일 기준의 지출입니다.

담당 기능과 한계

1일차의 도쿄 지역 패스, 2일차의 다른 지역 패스처럼 날짜별 추천과 적용 대상을 표시합니다. 운영사·노선·자격 조건이 해결되지 않으면 단순히 싸다는 이유로 추천하지 않습니다. 미확인 금액은 해당 일차에 별도로 남기고, 국내선 항공료는 지상 교통 패스 비교에서 분리합니다. 철도 기간 패스와의 순차 처리 한계는 앞 절과 같습니다.

8. 시간·날씨 보정: 경로 선택에 여행자의 현실 조건 반영

원래 방식의 역할

추정 모델은 확보하지 못한 값을 거리·속도·기본 요금 같은 입력으로 계산합니다. 가중 비용 보정은 후보를 비교하는 기준에 상황별 점수를 더하거나 빼는 방식입니다. 우리 구현에서 이들은 별도의 기계학습 모델을 훈련한 알고리즘이 아니라 규칙 기반 계산입니다.

우리 앱에 맞춘 가공

확보한 노선 정보와 요금을 우선 사용하고, 미확인 구간에는 교통수단별 거리 구간 모델로 시간과 요금을 보완합니다. 버스와 철도에 같은 기본 요금을 적용하지 않고, 장거리와 근거리에 서로 다른 규칙을 사용합니다. 여기에 이동시간을 1.2배로 늘린 뒤 분 단위로 올려 환승과 실제 이동의 여유를 반영합니다. 저장된 적용 비율을 검사해 같은 시간에 20%를 반복 적용하지 않습니다.

날씨는 방문 예정 시각의 일본 시간대에 맞는 3시간 예보를 사용합니다. 강수확률과 악천후 상태에서 최대 60점의 위험 가중치를 만들고, 실내 장소는 가산하고 실외 장소는 감점합니다. 방문 후보의 비용식에서는 날씨 적합도를 빼므로 적합도가 높은 후보가 유리해집니다. 예보가 없으면 점수는 0이며 오늘 날씨를 미래 여행 날씨로 대신 쓰지 않습니다.

가공 결과를 설명하는 예시

이동시간이 11분이면 11×1.2=13.2분을 올려 14분으로 일정에 반영합니다. 다시 조회해도 14분을 또 1.2배로 늘리지 않습니다. 비 예보가 있는 시간에 다른 조건이 비슷한 공원과 실내 전시관이 후보라면, 날씨 적합도에 의해 실내 전시관이 유리해질 수 있습니다. 다만 고정 일정이나 영업시간을 위반하면서 실내 후보를 넣는 것은 아닙니다.

담당 기능과 한계

시간이 너무 촉박한 일정과 날씨에 맞지 않는 방문을 줄이는 보정입니다. 평균 요금은 실제 승차권 견적과 같다고 보장할 수 없습니다. 시간이 보정되어도 좌표가 없던 실제 경로가 생기는 것은 아닙니다. 실제 경로, 평균 시간·요금, 날씨 점수는 각각 다른 정보와 기능입니다.

9. 발표에서 사용할 수 있는 설명

"우리 앱은 모든 장소를 거리순으로 연결하는 방식 대신, 도시 간 순서와 하루 안의 일정 배치를 나누었습니다. 도시 순서에는 Held-Karp 계열 동적 계획법을 사용하되 입국 공항에 가까운 선택 도시를 시작으로 고정하고 출국 공항까지의 비용을 반영하는 열린 경로로 바꾸었습니다. 하루 일정에는 탐욕 선택을 사용하되 영업시간, 식사 간격, 고정 일정, 숙소 복귀 마감을 먼저 검사하도록 가공했습니다. 다음 장소 평가에는 직전 이동을 되돌아가는 정도를 페널티로 추가했습니다. 교통 패스는 기간 선택과 적용 구간 집합 탐색을 결합해 중복 할인을 막고, 일별 비용과 적용 패스를 출력합니다."

"2-opt는 기존 구현에 내부 구간 교환과 총거리 제한을 붙인 형태로 남아 있지만 현재 생성 경로에서는 사용하지 않습니다. 따라서 현재 앱의 동작을 시간 제약을 반영한 2-opt라고 설명하지 않고, 제약 기반 탐욕 선택과 우회 페널티라고 설명합니다."

10. 분석 근거와 이전 설명 정정

이 설명은 현재 PC에서 사용하는 로컬 코드의 실제 조건과 호출 관계를 분석한 결과입니다. 알고리즘 이름은 구현의 구조를 설명하기 위한 분류이며, 특정 논문 전체를 그대로 구현했다는 의미는 아닙니다. 원형 알고리즘의 참고 문헌은 Held와 Karp의 순서 문제 동적 계획법 논문(1962, DOI 10.1137/0110015)과 Croes의 외판원 문제 논문(1958, DOI 10.1287/opre.6.6.791)입니다. Croes 논문 정보 (https://pubsonline.informs.org/doi/10.1287/opre.6.6.791).

앞선 문서에서 우회 페널티를 복귀 지점 기준이라고 설명한 부분은 직전 두 장소와 다음 후보 기준으로 정정합니다. 기존 장소 분할을 K-means라고 표현한 부분도 정정합니다. 해당 분할은 위도 또는 경도의 범위가 큰 축으로 정렬한 뒤 개수나 예상 활동시간을 기준으로 날짜별로 나누는 방식이며, 반복적인 중심 재계산과 군집 재배정을 하는 K-means 구현이 아닙니다.

검토 범위: 도시 방문 순서, 하루 일정 후보 선택과 제약 검사, 기존 2-opt, 패스 기간 선택, 적용 구간 집합 탐색, 일별 비용 산출, 날씨·이동시간 보정. 백그라운드 작업 큐와 알림은 실행 구조이므로 경로 알고리즘으로 분류하지 않았습니다.